# 경력기술서.

**김민혁** 프론트엔드 개발자

| | |
|---|---|
| **Phone** | 010-4373-7767 |
| **E-Mail** | jaqwe2301@gmail.com |
| **Portfolio** | bit.ly/4hWB6zr |
| **Github** | github.com/jaqwe2301 |
| **Blog** | mini-frontend.tistory.com |


## 경력 요약.

| | |
|---|---|
| **회사** | **Crewvation** (크루베이션) |
| **재직 기간** | 2024.05 ~ 2025.09 (**1년 5개월**) |
| **직무** | 프론트엔드 개발 |
| **담당 업무** | 프론트엔드 단독 개발, 제휴 기관 개발팀과의 연동 협의 |

## 1. LLM AI 기반 자기소개서 · 커리어 기록 첨삭 서비스

B2B2C SaaS · 초기 개발부터 상용화 · 운영까지

| | |
|---|---|
| **서비스** | 자기소개서 · 커리어 기록 · 면접 준비를 AI로 첨삭하고 관리하는 서비스 (개인 사용자 + 대학 · 행정기관 제휴) |
| **기간** | 2024.05 ~ 2025.09 |
| **역할 · 기여** | **프론트엔드 단독 개발** (사용자 서비스 전 화면 · 제휴 연동 · 배포 운영) |
| **기술** | TypeScript, Next.js 14 (App Router), TanStack Query, Tailwind CSS |

### 주요 업무

- **화면 · 기능 개발**
  - 자기소개서 · 커리어 기록 · 면접 준비 · 기업 공고 · 프로필 등 **서비스 전 화면** 개발
  - **Selection · Range API**로 드래그한 문장의 원문 위치를 계산해 첨삭 결과만 교체, 단어 단위 추가 · 삭제 하이라이트로 변경 전후 비교가 가능하도록 구현
  - 면접 준비의 카테고리 > 질문 > 꼬리 질문으로 이어지는 계층 구조에서, 각 단계의 순서 변경과 이동을 **드래그 앤 드롭**으로 구현
  - **토스페이먼츠**로 결제 · 쿠폰 기능 구현, **UTM 파라미터**로 유입 경로 추적
- **렌더링 · 데이터 동기화**
  - 면접 질문 목록에서 컴포넌트는 **memo**로, 핸들러 함수는 **useCallback**으로 참조를 유지해 불필요한 리렌더링 최소화
  - 자기소개서 · 면접 질문 데이터를 **TanStack Query로 캐싱**하고, 수정 시 캐시를 직접 갱신해 재조회 없이 화면 동기화
  - **디바운스 자동 저장**으로 변경 여부를 감지해 변경분만 저장하고, 페이지 이탈 경고로 작성 중인 글 유실 방지
- **UI 구조 · 웹 표준**
  - **Atomic Design** 패턴으로 컴포넌트를 Atoms · Molecules · Organisms · Templates로 나눠, 재사용 구조와 컴포넌트 구성 기준 정립
  - 검색 노출이 필요한 페이지는 **SSR**로 렌더링하고, 시맨틱 마크업 · 메타 태그 · sitemap으로 SEO 구성, 개발 서브도메인은 noindex로 노출 차단
  - aria-label · 명도 대비 등 웹 접근성 고려, 모바일 · 데스크톱 반응형 구현

- **서비스 운영 · 인프라**
  - **Docker 멀티 스테이지 빌드**와 Next.js standalone 출력으로 배포 이미지 구성, **GitHub Actions**로 빌드 · 배포 자동화
  - **Docker · Nginx** 기반 배포 환경 관리, SSL 인증서 적용 · 갱신
  - **Nginx 리버스 프록시**로 메인 도메인 요청을 별도 서버의 서비스 소개 사이트로 라우팅해, 웹 애플리케이션과 분리 운영
  - **Sentry** 커스텀 태그 · 컨텍스트로 에러 발생 맥락을 보강해 운영 장애에 먼저 대응, **사후 에러 제보 비중 30% 감소**
  - **AI 도구**를 활용해 팀에 필요한 백엔드 기능 구현

### 주요 성과 상세

**01** **빌드 타임 정적 데이터 전환으로 학과 9,329개의 초성 검색 지원, 결과 표시 시간 약 96% 단축**

**배경**

- 학력 정보 입력 시 사용하는 학과 검색 기능
- 학과 정보를 주는 공공 API가 **완성된 글자로만 검색**되어 **초성 검색 미지원**
- 초성 검색을 위해 전체 목록을 받아 브라우저에서 검색하는 방식 필요

**대안 검토**

- ① 전체 목록을 받아 **TanStack Query로 캐싱**: 첫 검색 결과 표시까지 약 1.5초 (모바일), 새로고침 시 캐시 소실
- ② **빌드 시점 정적 데이터 생성**: 검색 시 API 요청 없이 클라이언트에서 검색 처리
- **② 선택 근거**
  - 모든 사용자에게 같은 공통 데이터로, 빌드 시점 생성에 적합
  - 외부 API 응답 속도와 무관한 검색 성능
  - 새로고침 후에도 재요청 불필요

**실행**

- 배포 빌드 시 학과 목록 생성일을 확인해 **한 달 경과 시에만 갱신**
- 공공 API 조회 → 학과명 정리 → **정적 파일 생성**
- **gzip 압축**으로 전송 크기 축소 (289KB → 약 51KB)
- 검색어 입력 시 서버 요청 없이 정적 데이터 기반 클라이언트 측 검색
- **Lazy Rendering(지연 렌더링)**으로 검색 결과를 점진적으로 렌더링해 UI 프리징 완화

**성과**

| 지표 | 개선 전 | 개선 후 |
|---|---|---|
| 검색 → 결과 표시 (모바일) | 1,537ms | **55ms (약 96% 단축)** |
| 검색 → 결과 표시 (PC) | 729ms | **41ms** |
| 입력 반응 시간 (모바일) | 198ms | **78ms** |

**02** **대학 · 행정기관 제휴 가입 연동으로 복수 기관 B2B 계약 체결에 기여**

**배경**

제휴 기관 홈페이지에서 넘어온 학생의 **별도 입력 없는 가입 · 인증** 필요 (B2B 계약 요구사항)

**역할**

제휴 기관 개발팀과의 **전달 방식 협의**, 제휴 가입 흐름 **프론트엔드 구현**

**실행**

- 기관 코드 · 학번 · 단과대 · 학과를 **AES-256으로 암호화해 URL 파라미터로 전달**하는 방식 협의
- 카카오 로그인(OAuth) 동안 전달받은 값을 쿠키에 보관, 복호화 후 기존 인증 여부 확인
- **회원 상태(신규 · 인증 완료 · 인증 불가)에 따라 진입 경로 분기**

**성과**

- 제휴 기관 학생이 학교 정보를 다시 입력하지 않는 가입 흐름 완성
- **복수 기관 B2B 계약 체결**에 기여

---

## 2. 반려견 AI 채팅 · 위치 기반 커뮤니티 플랫폼

외부 인증 모듈 연동 · 리치 텍스트 에디터 · 지도 기반 콘텐츠 · 어드민 대시보드

| | |
|---|---|
| **서비스** | 반려견 AI 채팅과 위치 기반 커뮤니티를 제공하는 서비스, 운영용 어드민 포함 |
| **역할 · 기여** | **프론트엔드 단독 개발** (사용자 서비스 · 어드민) |
| **기술** | TypeScript, Next.js 15 (App Router), TanStack Query, Tailwind CSS |

## 주요 업무

- **화면 · 기능 개발**
  - **NICE 본인인증** 연동, **Next.js Route Handler**로 서버 측 인증 결과 수신 엔드포인트를 구현하고 요청 방식(GET · POST)별 분기 처리
  - **Tiptap 기반** 커뮤니티 글쓰기 에디터 개발, 이미지 리사이즈 · 링크 프리뷰 · 입력 제한 등 **커스텀 확장 7가지** 설계 · 구현
  - 실제 위치와 지도 탐색 위치를 별도 상태로 분리해, 탐색 좌표를 바꿔도 현재 위치가 유지되는 **Kakao Maps** 검색 흐름 구현
  - 줌 레벨 · 디바이스 크기에 따라 **지도 마커 크기를 동적으로 조정**해 탐색 가독성 개선, 모바일 · 데스크톱 반응형 대응
  - **차트 · 툴팁 인터랙션**으로 기간별 · 상태별 지표를 보여주는 어드민 대시보드 개발

## 주요 성과 상세

### 01 Next.js Route Handler 기반 인증 결과 처리로 일부 환경의 NICE 본인인증 실패 해결

**배경**
- **iOS Safari와 카카오톡 인앱 브라우저에서만** 본인인증 마지막 단계의 500 에러 발생
- 그 외 환경은 정상 동작해 개발 환경에서 재현 불가

**원인**
- 새 창 방식을 의심해 User-Agent 판별로 같은 창 전송을 시도했으나 같은 오류 발생
- **HTTP 메서드 · 페이로드 분석**으로 실패 환경은 결과가 **POST 본문(form data)**으로 도착하는 것 확인
- 결과 화면(page.tsx)은 GET 렌더링만 처리해 POST 요청에서 500 에러 발생

**대안 검토**
- ① 결과 화면(page.tsx)에서 수신: POST 본문 읽기 불가
- ② **Route Handler(route.ts)**에서 수신: 한 경로에서 GET · POST를 각각 처리 가능
- 전송 방식은 인증 기관이 정해 프론트엔드에서 선택할 수 없으므로 ② **선택**

**실행**
- Route Handler에서 GET · POST 핸들러를 각각 정의해 **요청 메서드별 분기**
- GET은 쿼리 파라미터, POST는 요청 본문(form data)에서 **본인인증 결과** 추출

**결과**
- iOS Safari · 카카오톡 인앱 브라우저 사용자의 **회원가입 본인인증 실패 해소**
- 인증 결과를 **서버 단 엔드포인트(Route Handler)에서 수신 · 처리**
- 전송 방식이 추가되어도 **메서드 핸들러 추가만으로 대응 가능한 구조**

관련 글 1 ↗ 관련 글 2 ↗

### 02 자체 에디터를 Tiptap 확장 구조로 재구성해 예외 처리와 유지보수 부담 축소

**배경**
- N사 블로그 에디터를 분석해 그대로 구현하는 것을 목표로 자체 에디터 제작
- 기능이 늘수록 줄바꿈 · 삭제 · 붙여넣기 등의 **예외 처리와 유지보수 부담 증가**
- React 상태와 직접 수정한 DOM이 어긋나 오류 추적 난이도 상승

**대안 검토**
- ① 자체 에디터의 예외 처리 계속 보강
- ② 문서 모델 기반 라이브러리 **Tiptap** 위에 자체 로직을 확장으로 이식
- 기본 편집 동작은 라이브러리에 위임하고, 기존 편집 로직은 확장으로 이식할 수 있는 **②로 전환 결정**

**실행**
- **이미지 리사이즈**: 드래그 중에는 화면만 갱신하고, 종료 시 한 번만 문서 데이터에 반영해 **불필요한 문서 갱신 방지**
- **링크 프리뷰**: 붙여넣은 URL을 감지해 메타데이터를 요청하고 카드 블록으로 자동 변환
- **입력 제한**: 변경 사항(트랜잭션)을 **문서에 반영하기 전에 검사**해 3,000자 초과 입력 차단
- **블록 배치**: 드롭 위치를 계산해 이미지 · 카드 블록 겹침 방지

**결과**
- 편집 상태의 **단일 출처를 문서 모델로 통일**해 React 상태와 DOM 불일치 해소
- 블로그 특화 기능을 **기능 단위 확장 7개로 모듈화**해 **기능 간 결합도 감소**
- 자체 에디터의 **비율 유지 리사이즈 계산을 그대로 이식**하고, 텍스트 노드 분할 방식의 서식 처리는 **서식 속성 기반으로 재구성**

관련 글 1 ↗ 관련 글 2 ↗